

An Introduction to Programming through C++

Abhiram Ranade

Ch 14: Arrays, part 2.

Arrays: the view so far

Defining an array:

```
elemtype  aname[asize];
```

Creates variables `aname[0]` , ... , `aname[asize-1]`

Each is of type `elemtype`

`aname` : name of array

Informally `aname` denotes the entire collection of variables

`aname[0]` , ... `aname[asize-1]`

Outline

The computer's view of arrays

- Where the elements are stored in memory

- How the computer indexes into an array

- What happens when an index out of range is used

Function calls using arrays

A function for sorting an array.

- Sort: rearrange elements so that they are in non-decreasing or non-increasing order

Computer's view of array definition

```
int q[4] = {11,12,13,14};
```

Assumption: a single `int` uses one four byte word

- 4 consecutive words of memory are allocated to `q`.
- Allocated memory used to store the variables `q[0]` , `q[1]` , `q[2]` , `q[3]` in order.
- Initial values stored in the allocated region.

Possible outcome

Address	Used for	Content
1004	q[0]	11
1008	q[1]	12
1012	q[2]	13
1016	q[3]	14

“Address”: address of first byte.

Address 1004: bytes 1004, 1005,
1006, 1007

Computer's interpretation of array name

Address	Used for	Content
1004	q[0]	11
1008	q[1]	12
1012	q[2]	13
1016	q[3]	14

“Address”: address of first byte.

Address 1004: bytes 1004, 1005,
1006, 1007

Array name

= address of allocated region

= address of 0th array element.

- For our array: `q = 1004`
- Type of `q` : `int *`
- Array name is a pointer, but its value cannot be changed. “`q = 1008`” is illegal.

In general

`elemtype aname[alength];`

Block of memory of length $S * \text{alength}$ is allocated,

S = size in bytes of a single `elemtype` variable.

`aname` = starting address of zeroth element = address of allocated block.

Value of `aname` cannot be changed.

Type of `aname`: `elemtype *`

Type of `aname[i]` : `elemtype`

Exercise

What is printed when the following executes?

```
int q[4]={11,12,13,14}, r = 2;  
cout << q << r << &r << endl?  
cout << q[0] << &q[0] << endl;
```

Make reasonable assumptions if you wish, or say if some value cannot be predicted.

Reasonable assumption: the compiler allocates memory in increasing order of addresses.

Exercise

```
int A[5];  
A[0] = 10;  
cout << A[0] << ' ' << &A[0] << ' ' << &A << endl;
```

What we discussed

How memory is allocated for an array: consecutive addresses

If an array has name `aname`, then

The type of the name `aname` = T^* where the array elements have type T .

The value of the name `aname` = addresses of the zeroth element of the array.

Next: How a computer interprets `aname [ index ]`



How does the computer interpret `aname[index]`

`[]` is a binary operator!

`aname`, `index` are the operands.

`aname[index]` means

The variable stored at $\text{aname} + S * \text{index}$, where S = size of a single element of the type `aname` points to.

Example: Next

Yes, the computer does a multiplication and addition to find the position of the element in memory.

Note that only a single multiplication and addition is done, however large the array is.

Example

Our old array q

```
int q[4];
```

Address Used for

1004-7 q[0]

1008-11 q[1]

1012-15 q[2]

1016-19 q[3]

q = 1004

type of q = int*

Computer's view of q[3]

q[3] :

variable of the type that q points to,

q has type int*, so q[3] has type int

stored at address $q + S * 3$ where S is size of a single variable of the type that q points to.

variable of type int, stored at $1004 + 4 * 3 = 1016$.

Same as what we call q[3]

Summary: How a computer gets to `aname[index]`

- The index is multiplied by the element size and added to the starting address to get the position in memory where the variable is stored.
- That variable is used.

Index out of range

Our old array q

```
int q[4];
```

Address Used for

1004-7 q[0]

1008-11 q[1]

1012-15 q[2]

1016-19 q[3]

q = 1004

type of q = int*

Suppose we execute:

```
q[10] = 34;
```

Mechanical interpretation as per our rule:

variable of the type that q points to, stored at address q + 10*S where S is size of a single variable of the type that q points to.

variable of type int, stored at $1004 + 10*4 = 1044$.

34 will get stored in address 1044 which is not part of q!

Possibly some other variable will be written into!

Summary

If you read or write from an improper address such as 1044:

You may store data into some wrong place.

You may get data from a wrong place.

Occasionally, the addressed may have been deemed “protected” – then your program may abort.

So make sure index is in correct range!

More remarks

Some programming languages prevent index out of range by explicitly checking.

- First the value of the index is checked to see if it lies in the range $0..size-1$.

- If it does not, an error message is printed and the program stops.

Index checking is not done in C++

- because it takes extra work, and

- because C++ designers believe that it is the programmer's job to ensure that the index is in range.

Example

What does the following code do?

```
int q[4];  
int* r;  
r = q;  
r[3] = 5;  
cout << q[3] << endl;  
cout << r[3] << endl;
```

The array `q` is allocated in memory.

Variable `r` is created.

`q` = address of the zeroth element of the array is placed in `r`.

Because `r`, `q` have the same value, `r[3]`, `q[3]` also denote the same variable.

What we discussed

- `aname[index]` is an expression with `[]` as operator.
- When the index is in range, the expression when evaluated, tells what variable is meant.
- If index is out of range, then the expression does not denote a valid variable.
- Calculation happens fast, in time independent of the array length.
- `aname[index]` is a valid expression if `aname` is a pointer.
- Next: Arrays and function calls.

Function calls on arrays

We might like to write functions to:

find the largest value in the array

find whether a given value is present in the array.

find the average of the elements in the array.

...

We see how to write functions involving arrays next.

A program to find the average of elements in array

```
double avg(double* M, int n){
    double sum = 0;
    for(int i=0; i<n; i++)
        sum +=M[i];
    return sum/n;
}

int main(){
    double q[]={11,12,13,14};
    cout << avg(q, 4) << endl;
}
```

Let us first check if this is a syntactically valid program, never mind what it does.

The types of **the arguments to a call** must match the types of **the parameters**

The first parameter of avg has type `double*`

The first argument in the call is `q`, whose type is `double*`, because it points to the first element of a double array.

The second parameter is of type `int`, and `4` in the call is indeed an `int`.

A program to find the average of elements in array

```
double avg(double* M, int n){
    double sum = 0;
    for(int i=0; i<n; i++)
        sum +=M[i];
    return sum/n;
}

int main(){
    double q[]={11,12,13,14};
    cout << avg(q, 4) << endl;
}
```

On execution of `avg(q, 4)` in `main`

Activation frame created for `avg`.

Value of `q` (starting address of array) copied into parameter `M`.

In each iteration, `M[i]` is needed.

`M` has value = starting address of `q`, and type `double*`

`M[i]` means “Variable of type `double` at address `q + 4*i`”

Thus `M[i]` in `avg` means `q[i]` of `main`.

Thus average of the elements of `q` is calculated in `avg`.

How arrays are passed to functions

Pass two arguments:

Starting address of the array, corresponding parameter is A, suppose.

Number of elements in the array, n.

Can elements be accessed in called function?

The expression $A[i]$ can be used in the called function to access the i th element because of how $[]$ works.

Array elements are not copied. If the function modifies the elements, then the modification will be seen in the calling program.

Remarks

When you pass the name of an array in a function call, its value, the starting address of the array, is copied to the corresponding parameter.

But because we pass the starting address of the array, we are effectively enabling the function to pass the array.

Can say

“Array name is passed by value”

“Array elements are passed by pointer”

The interesting part is the `[]` operator: given the address of an array and an index it can get us to the corresponding element, even if the address belongs to a different activation frame.

Remarks

An alternate syntax is also allowed.

```
int avg(double M[], int n){...}
```

In this `double M[]` is synonymous with `double* M`, but slightly more indicative that we expect `M` to be an array name, and not just any old pointer.

The second argument to `avg` is not “required” to be the array length. If it is smaller, then the function will return the average of just that part of the array.

What we discussed

No special mechanisms are needed to pass an array to a function.

We pass a pointer to the zeroth element, i.e. we state where the array begins in memory.

We pass the number of elements.

The `[]` operator can be used to access the individual elements of the array.

Next: Function to sort an array.



Sorting an array

Input: an array containing numbers.

Goal: rearrange the numbers so that they appear in non-decreasing order.

Example:

Array initially: 35, 12, 29, 70, 18, 29

Desired order: 12, 18, 29, 29, 35, 70

Sorting an array (contd.)

Sorting is an important operation. Chapter 16 gives a clue why.

There are many algorithms for sorting. Chapter 16 will discuss a clever and fast algorithm.

Here we discuss a slow, but easy to understand algorithm:

Selection Sort

Selection Sort

Basic idea:

- Find the largest number.

- Exchange it with the element in the last position.

- We have made progress:

 - the last position now contains the largest, as we would like it to.

- Now we can apply the same idea to the first $N-1$ elements of the array, where $N = \text{length of the array}$.

- Then to first $N-2$ elements, and so on.

Finding the index of the largest element

```
int posOfMax(float A[], int L){  
    // Returns the index of the largest element in A.  
    // Invariant for iteration i:  
    //     maxIndex = the index of the max in A[0..i-1].  
    int i=1, maxIndex=0; // invariant holds.  
    for(i=1; i<L; i++)  
        if(A[maxIndex] < A[i]) maxIndex = i;  
    return maxIndex;  
}  
//Number of comparisons: L-1
```

The main function and main program

```
void selSort(float A[], int N){
    for(int i=N; i>1; i--){
        int maxIndex = posOfMax(A,i); // posOfMax thinks array only has i elements
                                     // exchange A[i-1], A[maxIndex]

        float maxVal = A[maxIndex];
        A[maxIndex] = A[i-1];
        A[i-1] = maxVal;              // exchange done!
    }
}

int main(){
    float a[6] = {35, 12, 29, 70, 18, 29};
    selSort(a, 6);
}
```

Remarks

`posOfMax` will perform $L - 1$ comparisons, where L = length of array passed to `posOfMax`.

`selSort` calls `argMax` for $L = N, N-1, \dots, 2$

So the number of comparisons =

$$N + N-1 + \dots + 2 = (N+1)(N)/2 \approx N^2/2$$

There exist better algorithms which perform far fewer comparisons and thus run faster.

Demo `selSort.cpp`

Variation

Express selsort as a recursive program.

Recursive selsort

```
void RselSort(float A[], int N){  
    if(N == 1) return; // base case  
    int maxIndex = posOfMax(A,N);  
    float maxVal = A[maxIndex];  
    A[maxIndex] = A[N-1];  
    A[N-1] = maxVal;    // exchange  
    RselSort(A, N-1);  
}
```

Concluding remarks

The name of an array can be used in a program without the index, its value is the address of the 0th element of the array.

Indexing into an array is to be thought of as an operation `[]` between the array name and the index.

We can “pass” an array to a function by passing the array name. This does not copy the values but merely copies the address of the 0th element. In addition the length of the array must also be passed.

Writing functions on arrays is useful, and this is a skill you must master.

Selection sort runs in time proportional to the square of the number of elements being sorted. Faster algorithms are discussed in Chapter 16.



Outline

- Representing textual data
- Two dimensional arrays
 -
- Accepting arguments to programs from the command line.

Textual data

- `char` type meant to store single letter.
- Array of `char` can be used to store sequences of letters, e.g. words, sentences, paragraphs, e.g. “India”, “C++ is nice”.
- More elaborate, and safer solution based on `string` data type will be discussed in Chapter 22.

Character Arrays

```
char name[10], address[50];
```

- Defines two arrays, of length 10 and 50 respectively.
- Can be used to store character sequences, or “character strings”.
- An array of length n may store strings of smaller length.

Standard protocol inherited from the C language:

- store characters in the string in the array starting from element 0.
- After all the characters are stored, store the character ‘\0’,
- ‘\0’ = Character whose ASCII value is 0. “null” character.
- Key idea: “Everything until ‘\0’ is a part of the string, not what comes later”
- No need to explicitly store the length of the string.

Character arrays with initialization

```
char n1[20]="Ajanta", n2[]="Ellora";
```

n1 will be created with length 20, and initialized as follows.

- 'A', 'j', 'a', 'n', 't', 'a' will get stored in n1[0] through n1[5].
- Finally '\0' will get stored in n1[6].
- The elements n1[7] through n1[19] will not be initialized.

n2 will be created of length 1 + the length of the string "Ellora".

- It will also be initialized to the string "Ellora" followed by '\0'.

Syntax is only for initialization, not assignment.

Reading into char arrays 1

Example char array: `char buffer[80];`

- First way to read into it:

`cin >> buffer;`

- Reads one word (white space terminated) , stores it into `buffer` , terminated by `'\0'`.

Suppose user types:

`C++ is nice` (newline terminated)

- Only `C++` would go into `buffer`.

Note: `cin >> buffer` does not mention length of `buffer` , only its starting address.

- If user types more than 80 characters, there will be “index out of range”.
 - So allocate a large enough array.
- Note: `cin >> buffer;` works only for char arrays.

Reading into char arrays 2

Example char array: `char buffer[80];`

- Safe way to read into it:

`cin.getline(buffer, 80);`

- Reads a line (terminated by newline) or at most 79 characters.
- Stores them in buffer, terminated by `'\0'`. **Safe.**
- Line read may contain spaces.

Suppose user types

`C++ is nice` (newline terminated)

- `C++ is nice` would go into buffer followed by `'\0'`.

Printing char arrays

Example char array: `char buffer[80];`

... Code to assign value to buffer ...

- How to print:

```
cout << buffer;
```

- Print `buffer` content till `'\0'`.
- `buffer` assumed to contain a `'\0'`.
- Length of array is not important.

Note: For other types of arrays above would print address.

Character array processing

- Usually, the array length is ignored, instead we “process all elements till we find ‘\0’ ”
- The array must contain a ‘\0’.
- Very common idiom for character array processing.

A simple program

- Write a program to read in a name and count the number of letters in the name.
- For simplicity we use the unsafe method of reading.
- Standard linear search.
 - Terminate if find '\0'.

```
int main(){
    char name[80];
    cout <<"Name: ";
    cin >> name;
    cout <<"You typed:"<<name<<endl;

    int L=0;
    while(name[L] !='\0')
        L++;

    cout <<"Your name has "
        <<L<<" characters"
        <<endl;
}
```

Exercise

- Write a program that reads a word and prints its reverse.
 - First find the length of the word and then examine the characters in the word from the last character.

What we discussed

- Text is stored in an array of char
- Length of text is not explicitly stored.
- After the last real character, a null character: `'\0'`, i.e. character with ASCII value 0 is stored.
- “Sentinel”.

Next: Functions that operate on character strings and character string constants



Functions on character strings

- Standard rule for passing arrays to functions:
 - Pass the array name, whose value is starting address of the array
 - Pass the array size, i.e. number of elements.
- Rule for passing char arrays:
 - Pass the array name, whose value is starting address of the array
 - No need to pass the array length, because we don't use it anyway!
 - The sentinel '\0' tells us where the array ends.

Function to determine if character x is present in a string c s t r

```
bool findchar(char x, const char *cstr){
    // determines if x occurs inside cstr.
    for(int i=0; cstr[i] != '\0'; i++){
        if(cstr[i] == x) return true;
    }
    return false;
}

int main(){
    char name[] = "India";
    cout << findchar('d',name) << endl;    // should print 1;
    cout << findchar('c',name) << endl;    // should print 0
}
```

Demo

- findchar.cpp

Shouldn't this also work?

```
bool findchar(char x, const char *cstr){  
    // determines if x occurs inside cstr.  
    // Note that length of array cstr is not specified - not needed.  
    for(int i=0; cstr[i] != '\0'; i++){  
        if(cstr[i] == x) return true;  
    }  
    return false;  
}
```

```
int main(){  
    cout << findchar('d',"India") << endl;    // should print 1;  
    cout << findchar('c',"India") << endl;    // should print 0  
}
```

- It does, but how?

Character string constant

- The string is stored in some place in memory.
 - A null terminator is used.
 - It resembles a char array!
 - C++ denotes a character string constant by the address where it starts
-
- Just like any array!
 - Its type is `const char*`
 - `const` because you don't want it modified

A function to compare strings using Lexicographic/ Dictionary order

- Words are stored in the dictionary in a certain order that makes it easier to locate a word if need.
- All words beginning with 'a' stored first,
- then those beginning with 'b',
- then those beginning with 'c',
- ...
- Among words beginning with 'a' we store first
- those with second letter 'a', then
- those with second letter 'b', ...
- ...

Specification

Write a function that takes two character strings and returns

- '=' if they are equal,
- '<' if the first string will appear before the second in the lexicographic order,
- '>' if the first string will appear after the second in the lexicographic order.

Solution

```
char compare(const char *a, const char *b)
/*
Start from the beginning of both and compare corresponding
letters.
If they are same move on to next letter of both.
If at any point you find a's letter smaller, then return '<'.
Likewise if b's letter.
If both strings end without finding a difference return '='.
If one string ends, its letter will be '\0' which is smaller than
any other letter, so this case will be dealt with above.
*/
```

Solution (contd.)

```
{
    int i=0;
    while(true){
        if(a[i]=='\0' && b[i]=='\0')
            return '=';
        if(a[i]<b[i]) return '<';
        if(a[i]>b[i]) return '>';
        i++;
    }
} // more compact than the book.
```

Exercise

- Write a function which takes as argument a character string and returns 0 if the string is “zero”, 1 if it is “one”, ..., 9 if it is “nine”, and -1 otherwise.

What we discussed

- How character strings are passed to functions
- How constant character strings are represented and passed to functions.
- Notion of lexicographic order and a function to compare strings by lexicographic order.

Next: two dimensional arrays



Two dimensional arrays

Matrices or tables can be stored using “two dimensional arrays”

```
double xyz[m][n];
```

- Creates $m \times n$ variables: $xyz[i][j]$, for $0 \leq i < m$, $0 \leq j < n$.

- i th row of xyz :

```
xyz[i][0], xyz[i][1], ... xyz[i][n-1]
```

- j th column of xyz :

```
xyz[0][j], xyz[1][j], ... xyz[m-1][j]
```

- m, n : first, second dimension of array xyz .
- Variables stored in memory in “row major” order:
 - row 0, followed by row 1, ..., row $m-1$.

Two dimensional arrays (contd)

- Initialization possible:

```
int pqr[2][3]={ {1,5,7},  
                {13,6,2} };
```

- Values picked up from the initialization list in row major order.
- Better versions of two dimensional arrays are discussed in Chapter 22.

Example 1

- Create a 10x10 matrix A and initialize it to identity, i.e. value 1 in $A[i][i]$ for all i and 0 elsewhere

```
double A[10][10];  
for(int i=0; i<10; i++)  
    for(int j=0; j<10; j++)  
        if(i == j) A[i][j] = 1;  
        else A[i][j] = 0;
```

Example 2

- Create an array M to store marks of 10 students in 5 tests. Read the marks and store them in M.

```
double M[10][5];
for(int i=0; i<10; i++){
    cout <<"Give marks of student "
         <<i<<" : ";
    for(int j=0; j<5; j++)
        cin >> M[i][j];
}
```

Example 3

```
char countries[3][20] = {"India", "Nepal", "China"};
```

- Creates array `countries` with 3 rows each of 20 characters.
- 2d array = collection of 1d arrays.
- Thus `countries[i]` = array consisting of `countries[i][0]...countries[i][19]`
- Row `i` of array is initialized using `i`th character string in rhs.
- As usual, each character string implicitly includes a `'\0'` following it.
- Individual characters can be accessed in the usual manner, e.g. `countries[1][1]` will be `'e'`.

```
cout << countries[2] << endl; // prints "China"
```

Exercise

- Write a program that computes matrix C = product of two 10x10 matrices A , B .

Passing 2 dimensional arrays to functions

- Would be nice to have functions for matrices, tables, collections of strings...
- Example:
 - a function which multiplies an $m \times n$ matrix by an $n \times p$ matrix for any m, n, p
- The natural idea would be to pass
 - The starting address of the array
 - The number of rows
 - The number of columns.
- Unfortunately, C++ requires to specify the number of columns as a fixed number at the time of writing the function!
- This is a C++ language limitation.
 - Matrix multiply function above not possible
- Later we will see a more modern way which does not have this limitation.

Example

```
void printCountries(char c[][20], int n){  
    // We need to specify constant number of columns.  
    // The number of rows can be specified; it is n.  
    // This function simply prints all the strings in c.  
    for(int i=0; i<n; i++)  
        cout << c[i] << endl;  
}  
int main(){  
    char countries[3][20]= {"India", "Nepal", "China"};  
    printCountries(countries, 3);  
}
```


What we discussed

- Basics of two dimensional arrays
- Can be used for storing tables, matrices, multiple character strings.
- A two dimensional array can be regarded as an array of one dimensional arrays.

Next: command line arguments



Command line arguments

- So far, we have executed our programs from the command line by typing `./a.out` at the shell prompt.
- It is possible to supply arguments also from the command line.

For example, we can type

```
./a.out Math Biology
```

- The program can get access to the arguments we type. **Next.**
- Aside: `a.out` is just the default name given by the compiler, we can ask it to give a different name or rename the file if we wish.

A program that prints the command line

```
int main(int argc, char *argv[]){  
    for (int i=0; i<argc; i++)  
        cout << argv[i] << endl;  
}
```

- We call `main` with a more elaborate signature as shown.
- `argc`: gives the number of “words” typed on the command line
- Interpretation of `char *argv[]`:
 - `char x[]` says `x` is an array of `char`
 - `char* argv[]` says `argv` is an array of `char*`.
- `argv` has length `argc`.
- Each element of `argv` is a pointer to each word typed on the command line.
- How this executes: next

Execution

```
int main(int argc, char *argv[]){  
    for (int i=0; i<argc; i++)  
        cout << argv[i] << endl;  
}
```

What happens when this program is invoked as `./a.out Math Biology`

- `argc = 3`
- `argv` is an array of 3 elements
- `argv[0]` contains the address of an 8 element char array containing “`./a.out`” followed by `'\0'`.
- `argv[1]` contains the address of a 5 element char array containing “`Math`” followed by `'\0'`.
- `argv[2]` contains the address of an 8 element char array containing “`Biology`” followed by `'\0'`.

Concluding Remarks

- Later we will study the `string` class which is nicer and safer.
- However you must know `char` arrays because basic ideas in `string` are similar.
- Also because `char` arrays are used a lot in C language code, which you may encounter.
- More dimensions: add indices: `int a[10][20][30];`
- Later we will study a better version of multidimensional arrays.
- What we studied in this chapter will be needed in the better versions too.
- Also you may encounter simple multidimensional arrays in old programs written in the C language.
- Learn to use them.

